A Proposal for a System Definition Ontology for the Knowledge Hub

Draft 0.2 - 30 October 2026 - Ontology and proposed Rust library with a default file backend; adoption and implementation remain pending.

1. Purpose and proposed outcome

This proposal defines how a Knowledge Hub could represent the knowledge needed to understand, design, implement, operate and evolve an information system. It connects business intent, user experience, domain meaning, functional behaviour, information structures, visual interfaces, architecture, implementation and verification through explicit, traceable relationships.

The proposed outcome is a storage-independent System Definition Ontology: a vocabulary of identifiable objects, their properties, the relationships between them and the rules that make a model coherent. The ontology can be implemented in a graph database, an RDF store, a relational database or validated files without changing its fundamental meaning.

Section 21 proposes a concrete realisation as a Rust library named `knowledgehub`. Its first backend uses Markdown documents, graph JSON and versioned file artifacts. MongoDB and PostgreSQL adapters are future options outside the first implementation; their required semantics are addressed in the API and backend contract from the outset.

The central principle is:

> Represent concepts, behaviours, rules, representations and evidence as identifiable, versioned objects connected by typed relationships, with provenance attached to both objects and relationships.

A document, diagram or mockup can describe several objects. An object can appear in several documents and views while retaining one identity. The Knowledge Hub preserves these connections so that a reader or tool can move from a business objective to the behaviour, interface, implementation and evidence that support it.

This is a proposed synthesis of domain-driven design, experience design, requirements engineering, behavioural modelling, provenance and delivery assurance. It is not presented as an established DDD ontology. Established standards provide terminology and specialised representations where applicable; the combined model and its integration rules are the proposal made here.

2. The Knowledge Hub and its boundaries

The Knowledge Hub is the central, governed repository of system-related knowledge. Its logical contents include the system definition, original source references, interpretations, model revisions, representations, implementation references and evidence of verification and acceptance.

A Knowledge Hub can contain one or more system models, each with an explicit scope and owner. A model may describe a single application or a solution containing several applications, services and external-system references. Shared concepts are connected through qualified references with clear ownership. Alternative

designs, historical states and observed deployments are represented through explicit baselines rather than mixed into one ambiguous current model. An initial implementation can exercise one coherent system model; federation between independently governed Knowledge Hubs is a separate extension.

The Knowledge Hub is not required to become the production transaction store, source-code repository, document editor or observability platform. It can retain versioned references to those systems and capture evidence needed to explain a decision or result. Original files and native execution formats can remain in their appropriate systems.

Three distinctions guide the design:

Concern	Responsibility
Ontology	Defines available types, relationship meanings and constraints.
Logical data model	Organises identities, revisions, assertions, source anchors, baselines and specialised definitions.
Physical storage and tools	Persist, query, edit, render or execute those definitions through particular technologies.

The model should be sufficiently precise to support implementation while remaining independent of a particular frontend framework, programming language, database or deployment environment.

3. Principles

1. **Stable semantic identity.** Names, filenames, positions and diagram coordinates do not identify a concept. Renaming or moving a representation must preserve the identity of what it represents.
2. **Explicit meaning and ownership.** Types, relationships and consequential rules have clear definitions and owners. A familiar word can have different meanings in different contexts.
3. **One authoritative definition per scope.** A field or relationship has one declared authoritative definition within a baseline. Narrative and structured forms must not become competing sources of truth.
4. **Versioned provenance.** Definitions and assertions retain the exact source revisions, anchors, interpretation and review on which they depend.
5. **Complementary perspectives.** Business, experience, domain, behaviour, information, visual and engineering models are connected views of the system. Their relationships are many-to-many.
6. **Optional channels and profiles.** A background process does not need an invented screen, a pure calculation does not need an invented aggregate, and a system without AI does not need an AI profile.
7. **Behaviour beyond labels.** Functions, transformations and interactions include contracts and consequential semantics, not just names and arrows.
8. **Evidence with explicit scope.** Intended behaviour, implemented behaviour, verified results and human acceptance remain separate facts.
9. **Storage independence.** Files, tables and graphs are alternative physical representations of the same governed logical model.
10. **Progressive formalisation.** The model can begin with reviewed descriptions and gain executable contracts and constraints where uncertainty or consequence warrants them.

4. Three levels of representation

The ontology must distinguish the language used to model a system, the definition of a particular system, and occurrences within that system.

Level	What it contains	Generic service-scheduling example
Ontology	Modelling types and permitted relationships.	`AggregateDefinition`, `CommandDefinition`, `EventDefinition`, `ScreenDefinition`.
System definition	A particular system expressed through those types.	The Appointment aggregate, RescheduleAppointment command, AppointmentRescheduled event and rescheduling interface.
Execution and evidence	Particular instances, executions, deployed artifacts and observed results.	Appointment 123, an accepted rescheduling operation, an emitted event or a test execution.

Defining an `Appointment` entity type does not require importing every operational appointment into the design graph. An operational record may remain in its native store and be referenced through a suitably authorised, versioned locator.

This distinction applies consistently to event definition versus event occurrence, process definition versus process execution, transformation definition versus transformation run, and test specification versus test execution.

A reusable UI component definition is also distinct from its placement in a screen design and from an instance rendered in a running application. These identities can be connected without conflating their lifecycles.

5. Shared identity, revision and provenance kernel

All modelling families use a shared kernel. Specialised types add properties and constraints; they do not replace identity, revision or provenance with their own unrelated mechanisms.

Kernel object	Responsibility
`KnowledgeHub`	Identifies the governed knowledge repository, its scope, participants and authority policies.
`SystemModel`	Identifies the coherent solution definition held by the Hub.
`OntologyDefinition`	Identifies a version of the vocabulary, profiles and validation rules.
`ElementTypeDefinition`	Declares a modelling type, its properties, permitted specialisations and applicable constraints.
`RelationshipTypeDefinition`	Declares a predicate, its direction, endpoint types, qualifiers and multiplicities.
`ConstraintDefinition`	Describes a rule used to validate a record, subgraph or selected baseline.
`ProfileDefinition`	Selects types, relationships and completeness obligations for a modelling purpose.
`ElementIdentity`	Gives an element a stable ID, namespace and declared modelling type.
`ElementRevision`	Preserves an immutable version of the element's definition and references.

Kernel object	Responsibility
`RelationshipAssertion`	States a typed relationship between identified elements, with qualifiers, provenance and its own revision history.
`SourceArtifact`	Identifies an original document, interview, schema, code artifact, mockup, image, log or other source.
`SourceRevision`	Identifies an exact source snapshot through a content hash, commit, version or equivalent immutable reference.
`SourceAnchor`	Locates the relevant section, stable requirement ID, table row, JSON pointer, code symbol, UI element, image region or time range.
`Interpretation`	Records how source material was interpreted, by whom or what, and its proposed or reviewed status.
`ModelBaseline`	Selects a consistent set of exact element, relationship, ontology and applicable artifact revisions.
`RepresentationBinding`	Connects a model revision to a document fragment, diagram element, UI artifact, schema or implementation representation.
`ReviewDecision`	Records an assessment of an explicitly identified scope and revision.

5.1 Identity and revision rules

An identity survives a rename or a change of location. A revision captures a particular meaning at a particular point in its history. Published revisions are immutable; a change creates a successor revision. If a change introduces a genuinely different concept, it receives a new identity connected through an explicit relationship rather than silently replacing the earlier concept.

Relationships have identities and revisions as well. A modelling statement such as "this interaction issues this command" must remain inspectable after either endpoint changes. A baseline resolves its relationships against exact endpoint revisions, with any explicit pinning preserved. It must not silently combine incompatible revisions selected from different baselines.

Baseline membership is a set of explicit version bindings, not simply a timestamp or a label such as latest. Parentage, alternatives and comparison between baselines are recorded separately.

5.2 Provenance and authority

Provenance should capture both the source and the act of interpretation. A useful reference records source identity, immutable source revision, anchor, the statement being supported, derivation method, interpreter and review status. Evidence may support an individual property or relationship rather than an entire document or object.

Source status must distinguish observation, hypothesis, authored requirement, design decision, implementation observation and validation result. Confidence supplied by an extraction tool is not approval of its interpretation.

The Hub declares whether a representation is an authoritative definition, original evidence, implementation artifact or derived view. Existing document-owned definitions can remain authoritative while a structured model is proposed and reviewed. If structured authoring later becomes authoritative, the migration must explicitly replace the competing definition with a reference or derived representation.

Updating or refreshing a source introduces a new source revision. Earlier baselines retain the revisions on which they relied. Refresh policy is explicit; a changed external URL alone does not rewrite reviewed knowledge.

W3C PROV-O provides established concepts for entities, activities, agents and derivations that can be mapped to this provenance kernel. Its generic provenance use of the word entity is distinct from a DDD entity type. PROV-O.

6. Connected modelling profiles

The following profiles cover the principal aspects of a modern information system. They are parts of one connected model, with profile-specific constraints and optional applicability.

Profile	Principal object types	Purpose
Purpose and business	`Stakeholder`, `BusinessGoal`, `DesiredOutcome`, `BusinessCapability`, `BusinessFunction`, `BusinessRole`, `Measure`	Why the system exists, which work it supports and how success is assessed.
Requirements and reasoning	`Requirement`, `BusinessPolicy`, `BusinessRule`, `Assumption`, `OpenQuestion`, `DesignDecision`, `Rationale`, `Constraint`	What is required, why, and which decisions or uncertainties affect the design.
Experience	`Actor`, `Persona`, `Journey`, `JourneyStep`, `UserActivity`, `UserTask`, `UserStory`, `StoryMap`, `OutcomeCut`, `Scenario`	How participants pursue outcomes across interactions, alternatives and handoffs.
Strategic DDD	`Domain`, `Subdomain`, `BoundedContext`, `ContextRelationship`, `UbiquitousLanguageTerm`, `PublishedLanguage`	Meaning, responsibility and relationships between domain models.
Tactical DDD	`AggregateDefinition`, `EntityType`, `ValueObjectType`, `DomainService`, `Invariant`, `RepositoryPort`, `LifecycleDefinition`	Identity, consistency boundaries, domain rules and domain behaviour.
Application behaviour	`UseCase`, `CommandDefinition`, `QueryDefinition`, `FunctionDefinition`, `Parameter`, `Precondition`, `Postcondition`, `Effect`, `ErrorDefinition`	Requests, calculations, application orchestration and observable results.
Events and processes	`EventDefinition`, `EventOccurrence`, `EventHandler`, `ProcessDefinition`, `ProcessStep`, `ProcessExecution`, `State`, `Transition`, `Timer`, `Compensation`, `CorrelationPolicy`	Facts, reactions, coordination, time and recovery.
Information and transformations	`DataSchema`, `FieldDefinition`, `DataType`, `FieldConstraint`, `ProjectionDefinition`, `TransformationDefinition`, `TransformationRun`, `MappingRule`, `MigrationDefinition`, `DataStoreDefinition`	Information structures, derivation, mappings and persistence.
Interaction and visual design	`ScreenDefinition`, `ViewState`, `ComponentDefinition`, `ComponentPlacement`, `Interaction`, `NavigationTransition`, `DataBinding`, `ActionBinding`, `LayoutConstraint`, `DesignToken`, `Asset`, `AccessibilityRule`	Visible composition, interaction and bindings to system behaviour.
Architecture and integration	`ApplicationSystem`, `SoftwareComponent`, `Module`, `Interface`, `Port`, `Adapter`, `Endpoint`, `MessageChannel`, `ExternalSystem`, `IntegrationContract`, `Dependency`	Where responsibilities are implemented and how systems communicate.

Profile	Principal object types	Purpose
Security and trust	`PrincipalType`, `Role`, `Permission`, `AuthorizationPolicy`, `TrustBoundary`, `DataClassification`, `RetentionPolicy`, `Threat`, `AuditRequirement`	Authority, disclosure, trust boundaries and protection obligations.
Quality and operations	`QualityScenario`, `ServiceObjective`, `Environment`, `DeploymentDefinition`, `DeploymentInstance`, `Resource`, `ConfigurationDefinition`, `ObservabilityRequirement`, `RecoveryProcedure`	Performance, availability, deployment, configuration, operation and recovery.
AI participation	`AgentRole`, `ModelConfiguration`, `PromptDefinition`, `ToolContract`, `ContextPolicy`, `RetrievalPolicy`, `RunLimit`, `EvaluationCase`, `HumanReviewBoundary`	Explicit capabilities, inputs, controls and evaluation for nondeterministic behaviour where applicable.
Delivery and assurance	`DeliverySlice`, `ChangeDefinition`, `ImplementationTask`, `ReleaseWorkPackage`, `Release`, `AcceptanceCriterion`, `TestCase`, `TestRun`, `EvidenceArtifact`, `AcceptanceRecord`	How intended behaviour becomes implemented, verified and accepted.

The catalogue is a proposed starting vocabulary, not a requirement to instantiate every type for every feature. Extensions should declare their semantics and constraints and connect to the kernel rather than introduce ungoverned node labels.

7. Business, experience and DDD relationships

Business capability, business function and application use case answer different questions. A capability identifies an ability the organisation needs. A business function identifies a responsibility or category of business work. A use case describes behaviour through which an actor or system achieves an outcome.

For example, maintaining service appointments is a capability, coordinating field service is a business function, and rescheduling a confirmed appointment is a use case. The business function can be supported through several use cases and channels.

A journey records progression through activities and handoffs. A story expresses a need and benefit within that progression. A requirement states a promise precisely, while a scenario supplies a concrete example and expected result. Story maps group these elements and identify useful outcome cuts without turning the journey into a technical task list.

DDD introduces model boundaries and consistency responsibilities. Bounded contexts own meanings and rules; they are not automatically processes, deployable services or screens. An aggregate protects a consistency boundary and identifies exactly one root entity. Root is a role within aggregate composition, not a second entity created to duplicate the root's identity. Reusable value-object types and application orchestration remain distinct from aggregate ownership.

The same word may require different types or qualified meanings. A decision can mean business decision logic, a design choice, a review outcome or a domain entity in the described application. A function can mean business responsibility or executable computation. Context and type identity prevent these meanings from being merged merely because their labels match.

These DDD foundations can be mapped to the terminology in Eric Evans's DDD Reference. The broader connections between experience, delivery and evidence are part of this proposed ontology.

8. Typed relationship catalogue

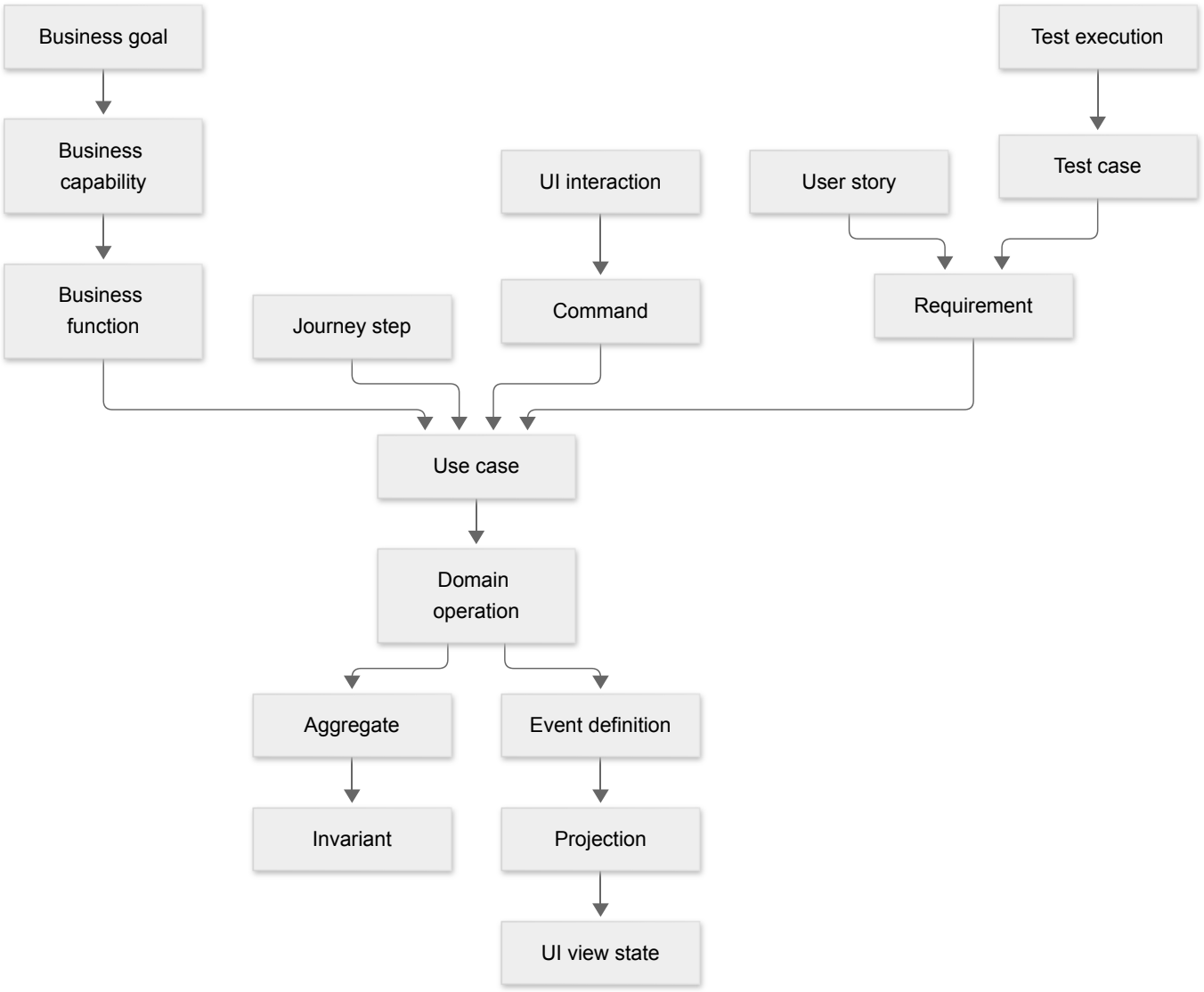
Relationship definitions are as important as object definitions. Every relationship type declares its allowed endpoint types, direction, multiplicity, qualifiers and applicable validation. Inverse labels may be shown in a view without storing a second competing assertion.

The following catalogue illustrates the core relationships. Multiplicities describe proposed modelling conventions and are interpreted within the selected profile and baseline. Partial models can have explicitly recorded coverage gaps before their completeness obligations are satisfied.

Source and relationship	Target	Meaning and constraints
`BusinessCapability contributesTo`	`BusinessGoal`	Connects an ability to its purpose; many-to-many.
`BusinessFunction realises`	`BusinessCapability`	Identifies supporting business responsibility; many-to-many.
`BusinessFunction decomposesInto`	`BusinessFunction`	Describes responsibility structure; declared decomposition must not form a containment cycle.
`Journey pursues`	`DesiredOutcome`	Identifies the participant outcome.
`Journey containsStep`	`JourneyStep`	Preserves step identity; sequence, alternatives and handoffs are explicit relationships, not inferred from list positions.
`UserStory supportsStep`	`JourneyStep`	Connects a need to the work being performed; many-to-many.
`Requirement formalises`	`UserStory` or `DesiredOutcome`	Connects a precise promise to its need; a requirement can support several stories and can exist without a user-story formulation.
`UseCase satisfies`	`Requirement`	Identifies behaviour intended to fulfil the promise; does not itself establish verification.
`Interaction issues`	`CommandDefinition`	Binds an interaction to an explicit request.
`CommandDefinition handledBy`	`UseCase`	Identifies the responsible logical handler in a command-handling profile; physical replicas are separate implementation objects.
`DataBinding reads`	`QueryDefinition`	Connects a displayed value or collection to an authorised query contract.
`UseCase invokes`	`FunctionDefinition`	Connects application orchestration to operations and calculations.
`FunctionDefinition operatesOn`	`AggregateDefinition`	Identifies affected domain state; functions without aggregate state need no such edge.
`AggregateDefinition hasRoot`	`EntityType`	Exactly one root within the aggregate definition.
`AggregateDefinition protects`	`Invariant`	Identifies its consistency obligations.
`ElementRevision ownedIn`	`BoundedContext`	Identifies domain ownership where applicable; cross-context similarity is represented by explicit mappings.
`FunctionDefinition declaresEvent`	`EventDefinition`	Declares a possible resulting fact, including the condition and commit boundary under which it can be published.

Source and relationship	Target	Meaning and constraints
`EventHandler consumes`	`EventDefinition`	Declares a reaction to a supported event contract.
`ProcessStep invokes`	`UseCase`	Connects process coordination to application behaviour.
`Transition triggeredBy`	`CommandDefinition` or `EventDefinition`	Identifies a transition trigger; its states and guards belong to the declared lifecycle or process.
`TransformationDefinition hasInputContract`	`DataSchema` or `ProfileDefinition`	Declares accepted input structure or model shape.
`TransformationDefinition hasOutputContract`	`DataSchema` or `ProfileDefinition`	Declares the required output structure or model shape.
`ProjectionDefinition derivesFrom`	`DataSchema` or `EventDefinition`	Identifies inputs used to construct a read model.
`SoftwareComponent implements`	`UseCase` or `Interface`	Connects intended responsibility to implementation structure.
`Adapter translates`	`IntegrationContract`	Identifies the boundary at which provider-specific meaning is mapped.
`AuthorizationPolicy governs`	`UseCase`, `QueryDefinition` or `Resource`	Makes access and admission rules inspectable independently of a UI control.
`TestCase checks`	`Scenario`, `Requirement` or `Invariant`	Connects verification intent to the promise being checked.
`TestRun executes`	`TestCase`	Refers to an exact test version and its observed execution context.
`EvidenceArtifact supportsCriterion`	`AcceptanceCriterion`	Associates results with a specific acceptance obligation.
`RepresentationBinding represents`	`ElementRevision`	Connects a document, diagram, UI or code representation to the exact definition represented.
`SourceAnchor supportsAssertion`	`RelationshipAssertion`	Records the evidence for a semantic relationship.
`RelationshipAssertion contradicts`	`RelationshipAssertion`	Preserves an identified disagreement until it is resolved.

A generic related-to link may be useful during exploration, but it does not substitute for a defined semantic relationship. A complete trace can cross several modelling families without forcing a one-to-one correspondence between their objects.



The diagram illustrates possible paths through the ontology. Its arrows abbreviate the typed relationships in the catalogue; they do not require every behaviour to have a UI, aggregate or event.

9. Functions and executable behaviour

The ontology distinguishes business functions, application use cases and software functions. It should also distinguish pure calculations from state-changing operations through declared effects and constraints.

Function-definition property	**Meaning**
Identity, revision and owner	Which operation is defined and who owns its meaning.
Input and output contracts	Typed parameters, results, units, reference types and structural constraints.
Preconditions	Conditions required before evaluation or admission.
Postconditions	Conditions guaranteed after successful completion.
Rules and invariants	Business and consistency obligations it must preserve.
Reads and effects	Data and resources it reads, modifies, produces or invokes externally.
Errors and partial outcomes	Named failure meanings, uncertain outcomes and preserved state.

Function-definition property	Meaning
Determinism	Whether equal inputs under the same declared dependencies must produce equivalent results.
Execution limits	Applicable timeout, resource, concurrency and cancellation constraints.
Behaviour body	A formal expression, decision table, algorithm description or versioned implementation reference.
Verification links	Relevant examples, tests, observed runs and acceptance obligations.

A label such as Calculate appointment duration does not fully define the calculation. The model must capture the relevant time semantics, inputs, result and errors, or point to a versioned authoritative definition that does so.

Commands represent requests to change state. Queries represent requests for information. Use cases establish application-level authorisation, orchestration and consistency boundaries. Domain operations preserve domain rules. Channels such as HTTP, messages, CLI commands, schedules and UI actions bind to these behaviours without redefining them independently.

Business decision logic can be represented through decision definitions, required inputs, rules, results and an expression or decision-table representation. A model review decision approving that logic is a separate governance object.

10. Event and process model

An event definition describes the meaning and contract of a fact. An occurrence records that fact in a particular execution. A handler defines the reaction to an event. These are separate objects.

An `EventDefinition` should identify its business meaning, owning context, schema version, possible producer, consumers, disclosure restrictions and compatibility policy. Its delivery contracts describe relevant ordering, duplicate delivery, retention and recovery assumptions.

An `EventOccurrence` should identify the event definition/version, event identity, subject identity/version, occurrence time, recorded time, causation, correlation and relevant producer or execution references. Sensitive payloads may remain in an authorised native store while the Hub retains safe references and evidence.

UI events, domain facts, integration messages and operational telemetry remain distinguishable. A button activation is not automatically a domain fact. An integration message can be derived from a domain fact through an explicit mapping and disclosure boundary.

A process definition composes steps, guards, branches, timers, event waits, outcomes and compensations. Its execution records the admitted steps and actual results under the versions used. Process state, aggregate lifecycle state and UI view state retain their separate meanings even when one influences another.

Event definitions do not select an event-sourced architecture. Publication after a committed transaction, recovery through an outbox or an external event stream are implementation mechanisms connected to the relevant contracts. The ontology records the selected mechanism and its evidence rather than inferring it from the presence of events.

11. Information and transformation model

The information model represents logical schemas, fields, datatypes, identifiers, units, nullability, reference constraints and lifecycle or retention properties. A domain entity definition, an API payload schema and a physical database table are distinct objects connected through explicit mappings.

Physical tables and columns are implementation evidence, not automatic aggregate or bounded-context definitions. A table may implement parts of several logical structures; a domain object may have several storage or transport representations.

11.1 Transformation kinds

Kind	Example	Required meaning
Data transformation	Convert a local appointment request into a normalised time-window value.	Input/output contracts, timezone and unit handling, validation, missing-value and error policy.
State transition	Apply a permitted status change after a guarded operation.	Subject lifecycle, source/target states, trigger, guard, effects and rejected-transition meaning.
Schema migration	Convert persisted appointment data between supported schema versions.	Supported source/target versions, data conversion, validation, interruption and recovery policy.
Model transformation	Convert an approved interaction model into a platform-specific UI model.	Source/target metamodel profiles, mapping rules, preserved semantics and information loss.
Artifact generation	Generate interface code, contract files or documentation from a baseline.	Generator version, selected inputs, output artifacts, provenance and required verification.

A transformation definition should declare input/output contracts, mapping rules, preconditions, postconditions, effects, errors, information loss and any claimed inverse or round-trip behaviour. Reversibility is established by an explicit contract, not assumed because two formats are related.

A `TransformationRun` records the exact transformation version, input revisions, parameters, execution environment, output revisions, result and diagnostics. Output-to-input trace mappings make derivation inspectable at the appropriate granularity, including individual fields or generated model elements where needed.

Model-to-model transformations and code generation can be represented without requiring every rule to be executable initially. A description remains a description until an executable representation and corresponding verification are linked.

12. UI and visual representation model

Visual representation is a first-class part of the proposed ontology. A screenshot or drawing alone cannot capture the interaction contract, while behaviour alone does not describe the intended visual experience.

Aspect	Model contents
Composition and appearance	Screens, regions, component definitions, placements, layout constraints, responsive rules, design tokens, typography and assets.
Interaction	Controls, gestures, form behaviour, navigation, view states, feedback, focus, keyboard operation and cancellation.

Aspect	Model contents
Semantic bindings	Query/data bindings, command/action bindings, schema/field mappings, permission conditions and validation rules.
Representation artifacts	Wireframes, HTML mockups, vector assets, screenshots, design files and rendered samples, linked to their exact model/source revisions.

A component definition describes a reusable component. A component placement identifies its use within a particular composition, including parameters, layout and bindings. A view state describes conditions such as loading, empty, denied, pending, conflicted, failed or successful, with the information and permitted interactions available in that state.

For a service appointment interface, the model could express:

```
Appointment edit screen
  contains -> New time-window form
  contains -> Save rescheduling control
  displays -> Appointment summary projection

Save rescheduling control
  issues -> RescheduleAppointment command

Appointment summary data binding
  reads -> GetAppointment query

New time-window form
  edits -> RescheduleRequest schema
  uses -> Time-window normalisation transformation
```

UI validation helps the participant understand and correct an input. Application and domain behaviour independently enforce the relevant rules for every channel.

The ontology must distinguish product layout from graph layout. UI layout constraints can be normative parts of the product definition. Coordinates used to arrange an ontology diagram are presentation metadata and do not change domain ownership or behaviour. A visual edge must declare whether it represents a semantic relationship or is merely a drawing connection.

13. Architecture, security, quality and AI participation

Architecture connects logical responsibility to software components, modules, public interfaces, adapters and external systems. Deployment definitions and instances then connect those implementations to environments, resources and configurations. A bounded context is not automatically a deployable service, and an interface is not automatically an HTTP endpoint.

Integration contracts describe native identity, schema, protocol, authority and error semantics. Adapters identify the transformations that cross these boundaries. API contracts and message contracts can be imported or generated as specialised artifacts while retaining their links to use cases and policies.

Security modelling includes principal types, roles, permissions, trust boundaries, authorisation and disclosure policies, audit requirements, retention obligations and relevant threats. Interface visibility is not admission authority: a hidden or disabled control cannot replace application-side enforcement. Secret material is

represented through controlled configuration references rather than copied into broadly readable model content.

Quality scenarios describe measurable conditions, stimulus, response and required limits. Service objectives, capacity assumptions, observability signals, recovery procedures and deployment constraints link to those scenarios. A measured result records the build, environment, workload and conditions under which it was obtained.

Where AI participates, its role, tool contracts, context selection, model/prompt versions, limits and human-review boundaries are explicit. A tool call invokes the same governed application behaviour as other channels. Evaluation cases and actual run evidence are linked to the relevant promises. Source content, model output or an agent role does not itself grant authority.

14. Delivery, verification and acceptance

The Knowledge Hub should connect product definition to delivery without treating planning records as proof of working behaviour.

For this proposal, a delivery slice means a bounded increment of working behaviour that delivers one coherent, demonstrable outcome and can be verified and reviewed as a unit. It identifies actor/trigger, observable outcome, complete path, applicable safeguards and evidence. This is a delivery convention, not a DDD modelling primitive.

A feature can require several slices, a slice can cross several contexts, and a slice can require several bounded changes. Implementation tasks describe the work inside those changes. Cross-cutting deployment verification and whole-release acceptance rehearsals can be represented as release work packages rather than forced into the product-slice category.

Verification specifications are distinct from their executions. A test case defines an example, inputs, steps and expected results. A test run records the exact test version, system baseline, build, environment, observed results and evidence. A suite execution groups test runs without erasing their individual scope.

The Hub should represent the following dimensions separately:

Dimension	Governing records
Definition and interpretation	Source references, interpretation records and reviews of exact model revisions.
Implementation	Versioned code/configuration artifacts and their declared implementation bindings.
Verification	Test executions, measurements, outcomes and evidence tied to the checked scope.
Acceptance	An authorised person's assessment of an exact scope, with conditions and limitations.

These dimensions cannot be safely collapsed into one generic status field. A reviewed requirement can remain unimplemented; an implemented function can fail verification; a passing technical test does not automatically establish human acceptance.

Current-state, target-design and observed-deployment baselines should also remain distinguishable. A proposed model change records its intended delta, rationale, affected elements, evidence and review. Accepted changes produce a new consistent baseline while preserving earlier knowledge and source bindings.

15. Worked example: rescheduling a service appointment

The following example illustrates the ontology through a hypothetical application. It describes intended behaviour and synthetic model records, not a running system or actual test result.

Model perspective	Example element
Business goal	Keep service commitments accurate and recoverable.
Capability	Maintain agreed service appointments.
Business function	Coordinate field service.
Actor	Authorised service coordinator.
Journey	Change an agreed appointment.
Journey step	Review and save a replacement time window.
User story	Reschedule a confirmed appointment while preserving its prior agreed time and attribution.
Requirement	Accept a permitted rescheduling only under current authority and revision checks, preserving the accepted history.
Use case	Reschedule an appointment.
Command	`RescheduleAppointment`.
Query	`GetAppointment`.
Aggregate and root	Appointment aggregate with Appointment as root entity.
Value object	TimeWindow with validated start/end instants and declared timezone interpretation.
Invariants	End follows start; only permitted appointment states can be rescheduled; accepted changes remain attributable.
Function	Validate the replacement window and apply an authorised revision-aware rescheduling.
Event	`AppointmentRescheduled`, published under the declared commit/publication contract.
Transformation	Normalise the entered local time window using explicit timezone and ambiguity rules.
UI	Appointment edit screen, time-window form, save control, conflict feedback and summary view.
Read model	Appointment summary projection supplied through an authorised query.
Process extension	Notify affected parties after the committed change, with its own delivery/recovery contract.
Verification	Normal, denied, invalid-time, concurrent-edit, duplicate-request and restart examples.

The application use case resolves current authority, validates the command and expected revision, obtains the required domain state, applies the operation and commits its state/history/publication intent under the selected consistency contract. The UI retains a rejected draft and explains a conflict rather than silently replacing another accepted edit.

Rescheduling may preserve the appointment's Confirmed lifecycle state while changing its time window. The model must not invent a new lifecycle stage for every operation. An explicit self-transition can be represented where a lifecycle notation requires it.

If notification is included in the selected outcome, it has a separately inspectable process and delivery result. A committed appointment update is not evidence that an external message was delivered. The interface and acceptance contract must state which result completes the selected outcome.

15.1 Illustrative element record

The following is a JSON serialization sketch. IDs identify hypothetical records. It is a fragment, not a complete export bundle or final schema; referenced owners, contracts, source revisions and anchors would be separately defined in a complete model.

```
{
  "id": "urn:example:command:reschedule-appointment",
  "type": "CommandDefinition",
  "revision": 1,
  "name": "RescheduleAppointment",
  "ownerContext": "urn:example:context:service-scheduling",
  "definition": {
    "inputContract": "urn:example:schema:reschedule-request@1",
    "resultContract": "urn:example:schema:reschedule-result@1",
    "operationIdentityRequired": true,
    "expectedRevisionRequired": true
  },
  "sourceReferences": [
    {
      "sourceRevision": "urn:example:source-revision:scheduling-requirements:1",
      "anchor": "urn:example:source-anchor:rescheduling-requirement",
      "claimKind": "authored-requirement",
      "interpretationStatus": "proposed"
    }
  ]
}
```

15.2 Illustrative relationship record

```
{
  "id": "urn:example:relationship:reschedule-command-handler",
  "type": "handledBy",
  "revision": 1,
  "from": {
    "elementId": "urn:example:command:reschedule-appointment",
    "revision": 1
  },
  "to": {
    "elementId": "urn:example:use-case:reschedule-appointment",
    "revision": 1
  },
  "sourceReferences": [
    {
      "sourceRevision": "urn:example:source-revision:scheduling-design:1",
      "anchor": "urn:example:source-anchor:rescheduling-use-case",
      "claimKind": "design-decision",
      "interpretationStatus": "proposed"
    }
  ]
}
```

The relationship can receive a review or a conflicting assertion independently of either endpoint. A complete baseline binds these endpoint revisions and the applicable relationship-type definition so that the assertion remains interpretable later.

16. Physical storage and interchange

The ontology should be defined before selecting its database implementation. Storage choices affect query, transaction and operational mechanisms, but should preserve the logical contracts.

The proposed initial implementation is the file backend described in section 21. The other mappings below explain how the same logical model can be preserved by future implementations; they do not expand the first implementation's backend scope.

Representation	Suggested mapping
Property graph	Typed element and revision nodes, typed relationships, and explicit assertion nodes when relationships require their own provenance, review or further relationships.
RDF	Identified resources, typed predicates and qualified assertions, with named model scopes or equivalent baseline representations.
Relational	Identity/revision tables, specialised definition tables, relationship assertions, source/anchor tables, review records and baseline membership.
JSON or files	Identified records with explicit typed references, selected versions and bundle-level validation.

A relational implementation could organise the shared kernel around element identities, element revisions, relationship identities/revisions, source artifacts/revisions/anchors, interpretations, baselines and baseline memberships. Type-specific tables or validated payloads then carry the specialised definition data. Native foreign keys and transaction boundaries remain valuable even when semantic relationships form a graph.

Import and export should identify ontology version, model identity, selected baseline, contained revisions, qualified external references and source bindings. An import validates endpoint resolution and profile constraints before promoting a candidate bundle. Partial or unresolved imports remain explicit rather than silently creating accepted dangling references.

A generic property bag does not by itself implement this ontology. Record schemas and graph-wide constraints are required regardless of physical storage. The Hub should also preserve specialised executable artifacts where translating them into generic fields would lose meaning.

17. Validation and completeness

Validation has three layers: record/schema validity, graph/baseline consistency and adequacy for the selected purpose. The first two can be largely deterministic. The third requires explicit coverage obligations and, where appropriate, practitioner or user assessment.

Rule	Required check
V-01 Identity	IDs are unique within their declared namespace; aliases and display names cannot replace identity.
V-02 Type conformance	Element properties and qualifiers satisfy the selected type/profile definitions.
V-03 Relationship endpoints	Endpoint types, direction and multiplicity satisfy the relationship definition; external references are explicitly qualified.
V-04 Revision integrity	Published revisions and their content hashes remain stable; baseline bindings resolve to compatible explicit versions.
V-05 Provenance	Required sources resolve to exact source revisions and meaningful anchors; unsupported interpretations are recorded as such.
V-06 Authority	Conflicting normative definitions are identified; the authoritative owner for the selected scope is explicit.
V-07 Domain ownership	Context ownership and cross-context mappings are valid; similarity of labels does not merge meanings.
V-08 Aggregate composition	Every complete aggregate definition has one root and explicit consistency obligations.
V-09 Behaviour contracts	Inputs, outputs, guards, effects, errors and implementation references meet the selected behaviour profile.
V-10 State and process structure	Transitions resolve to the correct lifecycle/process states; guards and effects are defined; prohibited cycles are rejected while legitimate loops remain possible.
V-11 Event compatibility	Producer, schema, handler and delivery contracts identify compatible versions and required deduplication/correlation semantics.
V-12 Transformation contracts	Inputs, outputs, mappings, declared information loss and execution lineage meet the selected transformation profile.
V-13 UI bindings	Actions, reads, field mappings and view-state conditions resolve to valid behaviour/schema/policy definitions.
V-14 Architecture mappings	Implementations, adapters and deployment references point to the responsibilities/contracts they claim to realise.

Rule	Required check
V-15 Evidence scope	Test executions and measurements identify exact definitions, builds, environments and observed results.
V-16 Review and acceptance	Review records bind their scope and revisions; document structure or generated artifacts cannot imply implementation or acceptance.
V-17 Completeness obligations	Every obligation required by the selected profile is satisfied or recorded as an explicit gap or deliberate exclusion.

SHACL provides a standard means of validating RDF graphs against declared conditions. Other storage forms can enforce equivalent rules through their own validators. SHACL.

Completeness is assessed for a purpose and baseline. A persistent mutation profile may require authority, validated input, state/invariants, concurrency, durable outcome and recovery examples. A UI profile adds interaction, bindings, visible states, responsive and accessibility obligations. A pure transformation profile may require neither a journey nor a screen.

Passing structural checks establishes that the model conforms to its rules. Whether the model captures the right business meaning, useful interaction and sufficient operational detail requires separately recorded assessment. A system becomes implementable through precise contracts, algorithms, representations and evidence connected to this structure, not through a large inventory of named nodes alone.

18. Standards alignment

The Knowledge Hub should connect to specialised standards through explicit mappings and versioned artifact bindings. It should preserve native semantics and record which representation owns each definition rather than assuming all formats are losslessly interchangeable.

Reference	Proposed role
DDD Reference	Domain vocabulary, context boundaries, aggregates and related modelling concepts.
W3C PROV-O	Provenance, responsible agents, activities and derivation.
W3C SHACL	RDF graph-shape validation where RDF is selected.
OMG BPMN 2.0.2	Specialised process models and their artifact bindings.
OMG DMN 1.5	Specialised decision models and decision-table representations.
OMG QVT 1.3	A reference for model-to-model transformation representations.
OpenAPI	Machine-readable HTTP interface contracts.
AsyncAPI	Machine-readable message-driven interface contracts.

The versioned references identify terminology and representation examples for this proposal. They do not prescribe a complete toolchain or require adoption of every standard.

19. Proposed adoption sequence and review criteria

The first formalisation should test a complete, useful path rather than attempt to instantiate every profile at once.

Stage	Proposed result	Review criteria
1. Kernel and vocabulary	Define identity, revision, provenance, authority, baselines, initial types and relationship constraints.	A rename preserves identity; original source revisions remain resolvable; incompatible or dangling relationships are rejected.
2. One complete system path	Model a neutral example across intent, journey, requirements, behaviour, domain, UI or another channel, events, transformations and verification targets.	Readers can traverse the path in either direction and explain the relevant normal, denied, concurrent and recovery cases.
3. Interchange and validation	Implement a bounded import/export and graph-consistency proof for the selected profiles.	Round-trip preserves identities, types, relationships, exact version bindings, source anchors and explicit gaps.
4. Rust library and file-backend proof	Implement the proposed `knowledgehub` library and its default Markdown/graph-JSON backend, with a defined authoring route.	Atomic edits, revision conflicts, idempotent retries, import/export and interrupted saves preserve the public API contract; database adapters remain deferred.
5. Profile expansion	Add specialised architecture, operations, decision, process or AI profiles as actual needs require.	Each extension defines its types, relationships, obligations, native artifact mappings and verification examples.

These stages describe a possible adoption sequence. They are not evidence that an implementation, migration or generated application already exists.

The proposal is ready for an adoption decision when the following questions have explicit answers: which initial profile is required; who owns each authoritative definition; how model edits are reviewed; which baseline is being described; and which results constitute a successful first implementation.

20. Decisions to resolve before implementation

1. **Authority and authoring:** which definitions initially remain in documents, which are structured records, and how changes across those representations are committed coherently.
2. **Initial profile:** the minimum set of types and completeness obligations needed for the first end-to-end example.
3. **Expression languages:** how predicates, decision logic, mapping rules and algorithms are represented, executed or referenced.
4. **File format and durability:** finalise the default backend's snapshot manifest, Markdown/graph-JSON ownership rules, supported filesystem profile, locking, durability and recovery procedure. Assess MongoDB/PostgreSQL transaction and query mechanisms only when those deferred adapters are authorised.
5. **Review governance:** who may propose, assess, accept or supersede a definition or relationship, and how disputed interpretations remain visible.
6. **External references:** how independent sources and Hubs are identified, authorised, captured and refreshed without silently changing earlier baselines.

7. **Execution evidence:** which native systems retain operational records and which immutable references or captured results the Hub needs to explain a verified claim.
8. **Presentation fidelity:** which UI, diagram and executable formats must round-trip, and where a mapping is intentionally partial or lossy.

The first adoption decision should select a small coherent model and its review criteria. Further breadth should follow demonstrated needs while preserving one identity, provenance and validation foundation.

21. Proposed Rust library `knowledgehub`

21.1 Implementation objective and scope

Implement a Rust library named `knowledgehub` that provides the official application API for reading, validating, changing, reviewing and exchanging Knowledge Hub content. The library owns the model's semantics and guarantees; storage adapters own the mechanisms used to preserve them.

The requested name identifies a new library. A crates.io registry lookup on 30 October 2026 returned that no crate named `knowledgehub` existed. Publication and registry-name availability must be checked again when a release is prepared. This proposal does not depend on an existing package or claim that a library has already been implemented. Registry lookup.

The first implementation includes the Rust library and one default file-based backend. It stores authored narrative as Markdown, explicit model elements and relationships as graph JSON, and supporting content as versioned file artifacts. It requires no database server.

MongoDB and PostgreSQL are planned alternative backends. Their drivers, configuration, deployment, operational support, performance work and actual database parity checks are outside the first implementation. The initial API nevertheless defines the snapshot, transaction, identity, query and portability semantics that those adapters would have to satisfy.

An HTTP service or remote SDK could later expose the same use cases. Neither a network service nor a particular application UI is required to establish the first library contract. Library encapsulation separates responsibilities; enforceable isolation from untrusted callers additionally requires appropriate process, credential and filesystem boundaries in its host deployment.

21.2 Rust structure and dependency boundaries

Begin with one crate and clear internal modules rather than a separate crate for every ontology profile. The proposed module responsibilities are:

Module responsibility	Owns
Public API and contracts	The `KnowledgeHub` handle, owned request/result types, stable identifiers, revision/snapshot references and structured errors.
Model and validation	Ontology/profile interpretation, definition ownership, graph constraints, baseline resolution and change validation.
Application use cases	Authorisation hooks, coherent reads, change admission, review-policy enforcement, provenance and commit orchestration.

Module responsibility	Owns
Interchange	Portable manifests, Markdown/graph-JSON bindings, source-artifact references and validated import/export.
Storage port	Snapshot-bound reads and guarded atomic commit requirements, independent of drivers and filesystem layout.
Default file adapter	File encoding, immutable snapshot publication, coordinated writers, durability and restart recovery.

The model and use cases do not import database driver records, native session objects, HTTP request types or filesystem transaction machinery. Backend construction belongs to the host's composition/configuration code. Ordinary callers operate on Hub/model IDs and public contracts, without depending on physical paths or collection/table names.

The first package can enable a `files` Cargo feature by default. Future database drivers may be isolated through optional features or separate adapter crates once implemented. Empty placeholder features must not be advertised as functioning adapters. Cargo features provide conditional compilation and optional dependencies; they do not define runtime backend capabilities. Cargo features.

Public I/O-bearing operations should be asynchronous from the outset so that later database or remote adapters do not require replacing synchronous consumer contracts. Before implementation, select and record the Rust edition, minimum supported Rust version, executor policy and supported host/filesystem profiles. Public data contracts should not require callers to handle executor-specific resources. Blocking filesystem work must follow the chosen executor's bounded blocking-I/O policy.

21.3 Public API proposal

The public API is defined by Knowledge Hub behaviour. It is not a thin wrapper around database CRUD or arbitrary JSON-path edits.

Capability	Proposed operations and guarantees
Model and ontology discovery	Enumerate available models, ontology versions, profiles, element types, relationship types and applicable constraints.
Consistent reads	Resolve exact element/relationship revisions, source anchors and baseline membership through an explicit snapshot reference.
Portable queries	Filter by declared types, relationships, scope and revision; use documented ordering, bounded traversal and opaque pagination cursors.
Change validation	Assess a proposed change set against a selected base and return structured violations; validation alone makes no accepted mutation.
Atomic changes	Commit related definition, document, relationship and provenance changes together under current revision and policy guards.
Baselines and history	Create, retrieve and compare exact version selections; preserve earlier definitions and references.
Review	Record assessments of exact scope/revisions and enforce any review required by the configured policy.

Capability	Proposed operations and guarantees
Artifacts and portability	Store or resolve versioned artifact content, export a portable bundle, and stage/validate imports before promotion.

The configuration layer may expose a file-backend constructor; this is distinct from the storage-independent consumer API. The following illustrates a possible call shape, not an implemented or final Rust API:

```
use knowledgehub::KnowledgeHub;
use knowledgehub::backends::files::FileBackend;

let backend = FileBackend::open("./knowledge").await?;
let hub = KnowledgeHub::builder().backend(backend).build().await?;

let receipt = hub.commit_change_set(context, change_set).await?;
```

Here `context` is supplied by the trusted host's actor/policy integration, and `change_set` is a typed request containing model/branch scope, an expected head, an operation identity and proposed operations. An ontology element describing a role does not itself grant the caller Hub privileges.

The committed result identifies the accepted commit, resulting head, affected element/relationship revisions and relevant change sequence. Stable errors distinguish authority denial, unavailable scope, invalid input, ontology violations, revision conflicts, operation-identity reuse and storage/recovery failures. Native driver errors remain internal diagnostics with safe correlation references.

Consumer code should not receive SQL connections, MongoDB sessions or mutable references into an in-memory accepted graph. The same consumer operations should remain valid when host composition selects a future supported backend.

21.4 Change sets and consistency guarantees

The initial atomic commit scope is one model branch. A change set may contain several related operations, such as revising a Markdown-owned definition, updating its structured references, adding a relationship and recording provenance. Multi-model atomic changes are a separate capability and must not be implied by several ordinary commits.

Every supported backend must preserve these core guarantees:

1. A change set either commits all accepted changes and required metadata or leaves the accepted head unchanged.
2. Published revisions remain immutable, and a snapshot resolves a consistent set of exact versions.
3. The operation's expected head and relevant revision guards are checked at commit admission, not merely during an earlier UI or validation pass.
4. Graph-wide constraints are protected against concurrent changes; independent record checks alone are insufficient where a rule spans several relationships or depends on absence.
5. The same operation ID, scope, actor and canonical request fingerprint resolve to the original result on retry. Reusing that identity for different intent is rejected.
6. No durable success acknowledgement is issued before the selected backend's commit/durability contract is fulfilled.

7. History, provenance, the new head and deduplication result are published coherently. Required change notifications are derived from durable committed records, not emitted as success beforehand.

The first implementation can serialise writers per model branch and use an expected-head guard. This deliberately favours a clear correctness boundary over fine-grained write concurrency. A future implementation can change its locking strategy while preserving the same observable outcomes.

After current access checks, an operation retry consults its deduplication record before treating the original expected head as a new stale write. Returning an existing receipt never re-executes the mutation. A caller that has lost access does not regain it through an old operation ID.

Validation can prepare a commit plan from a coherent base snapshot. The storage adapter applies that plan only if its guards still hold; otherwise the caller must inspect or rebase the change and validate again. The API must not blindly replay a stale user intention against a new head. External source acquisition and other network work occur outside the short commit-publication boundary.

21.5 Default Markdown and graph-JSON backend

The default backend should be a portable directory with an explicit format version. One possible layout is:

```
knowledge/
  hub.json
  refs/<model>/<branch>.json
  snapshots/<commit>/manifest.json
  snapshots/<commit>/documents/*.md
  snapshots/<commit>/graph/elements.json
  snapshots/<commit>/graph/relationships.json
  snapshots/<commit>/provenance/*.json
  snapshots/<commit>/baselines/*.json
  artifacts/<content-hash>/...
  indexes/...
```

This is an illustrative internal layout, not a required path convention for consumers. The snapshot manifest binds the exact ontology/profile versions, Markdown documents, graph records, provenance, baseline selections and content hashes needed to reconstruct that snapshot. The first implementation may use complete snapshot generations for small models; content-addressed sharing or incremental storage can follow without changing the public API.

Markdown documents carry readable narrative and any definitions declared to be document-owned. Stable document/fragment identities preserve references across heading or filename changes. Graph JSON carries explicit typed identities, relationships, revision bindings and structured definitions owned by the graph. The authority map determines which representation owns each field; a cached structured extraction of Markdown is marked as derived rather than becoming a competing normative copy.

An API edit to a document-owned definition must update that authoritative content and its required graph/provenance bindings coherently. Similarly, an edit to a graph-owned relationship must not be implemented only as a diagram or prose change. A diagram's layout remains separate presentation metadata.

Accepted snapshots are immutable. Direct editing of published snapshot files is outside the supported write protocol. Human editing can use an exported working copy or authoring workspace whose changes are detected, reconciled and imported through the same validation/change-set API. Imports reject malformed metadata, unresolved required references and stale-base edits rather than silently replacing the accepted model.

21.6 File publication and recovery

The adapter should coordinate the supported writers, construct and validate a candidate snapshot in staging, persist its content and commit metadata, and then publish the new branch reference using the selected filesystem's tested atomic replacement and durability procedure. Readers resolve a branch head once and remain bound to that snapshot for the read operation.

An interrupted publication before the head advances must leave the prior snapshot usable. After a successful head publication, the new snapshot must be complete and recoverable, including the original operation result. Incomplete staging content is never treated as an accepted baseline; unreferenced objects may be retained for inspection or reclaimed through a defined cleanup procedure.

Atomic rename by itself is not the complete durability or multi-file consistency contract. The implementation must specify file and directory persistence, writer coordination, supported filesystem assumptions and recovery after interrupted writes. A manifest must never point to missing content after an acknowledged commit within the supported profile.

Initially support a coordinated single-host writer profile, with concurrent readers on immutable snapshots. Multiple clients can later be served through one host service using the same writer coordination. Arbitrary multi-host writers sharing a network folder are outside the first file-backend guarantee.

Cancellation or a lost response near commit must be recoverable through the original operation ID. The absence of a response is not proof that the change did not commit. The caller needs an operation-status/result lookup rather than a new identity and an uncontrolled second attempt.

21.7 Storage interface and portable query semantics

The internal storage interface should expose snapshot-bound reads and guarded commits of validated plans. It must not require the application core to execute user callbacks inside a database-specific transaction or to manipulate live storage objects. Each adapter owns the transaction, lock, compare-and-swap and recovery mechanisms needed to meet the contract.

Rust traits can express this backend interface. If dynamic dispatch is selected, its asynchronous methods must use a compatible representation, such as explicitly boxed futures; a native `async fn` trait method must not simply be assumed usable through a trait object. The final internal interface should be tested before it is offered as a stable third-party adapter API. Rust trait interfaces and dynamic compatibility.

The portable query contract must define filtering, null/missing-value distinctions, ordering, pagination, relationship direction, traversal bounds and snapshot selection. A client should not send arbitrary SQL, MongoDB filter documents or a vendor graph language through the official API.

The initial file backend can satisfy bounded queries through scans and rebuildable indexes. Future backends can optimise execution while returning equivalent semantic results. Full-text ranking, vector search or other specialised facilities can be optional capabilities, with explicit availability and result provenance; core integrity guarantees are never optional capabilities.

Search indexes, embeddings and rendered views are derived. If updated asynchronously, their responses identify the baseline or indexing watermark they represent. Authoritative revision resolution and commit validation cannot silently use a stale derived index as their source of truth.

21.8 Deferred MongoDB and PostgreSQL adapters

Backend	Future internal mapping	Required semantic preparation now
MongoDB	Separate records for definitions/revisions, relationships, source content and model manifests, using supported transaction and concurrency mechanisms.	Owned portable values, asynchronous operations, graph-wide commit guards, atomic multi-record outcomes and no dependency on native document IDs or sessions.
PostgreSQL	Identity/revision, relationship, provenance and baseline tables, with suitable JSON payloads for profile-specific structures.	Explicit snapshot/commit semantics, stable public IDs, protected invariants, opaque cursors and no public dependency on SQL rows, connections or transaction callbacks.

MongoDB's single-document atomicity alone does not establish the required multi-record Hub commit. A future adapter must demonstrate the selected transaction/deployment configuration and prevent graph-wide write anomalies. PostgreSQL likewise requires an appropriate locking or isolation strategy; a transaction with unprotected reads is not automatically sufficient for every invariant. MongoDB transactions, PostgreSQL transaction isolation.

Markdown remains a logical document format across backends. A database adapter can store its content and metadata as versioned records without manufacturing a physical path for every document. Original artifact bytes, content hashes, stable anchors and baseline bindings must survive the storage change. Filesystem locations, SQL identifiers and native database document IDs remain implementation details.

The first release should contain no database-driver dependencies, placeholder working-adapter claims or database deployment requirements. Future adapters become supported only after actual-engine conformance results establish the same observable contract.

21.9 Interchange, API evolution and future service exposure

Keep the Rust API/crate version, ontology/profile version, file-format version and portable bundle version separate. A new ontology type or adapter should not silently change the meaning of an existing API operation. Structured errors and public contracts need an explicit compatibility policy appropriate to the crate's published version.

Export produces a backend-neutral bundle containing selected identities/revisions, relationship assertions, ontology/profile bindings, Markdown documents, source anchors, required artifact content or qualified external references, baseline selections and relevant review/history records. A complete backup/export profile must also declare how operation identities and deduplication history are retained for safe recovery and migration. A native database dump is not the portable interchange contract.

Import stages and validates its scope before promotion. Moving the same model from files to a future database backend should preserve meaning, identity, exact revisions, history and original source hashes; it need not preserve physical row order, native storage IDs or database JSON formatting. Semantic content hashing needs a declared canonical encoding, while original-source hashing refers to the retained original bytes.

A later HTTP service should call this library rather than reimplement its validation and commit rules. Its authentication, transport errors and remote-client behaviour are an additional adapter boundary. An embedded library API is the first deliverable; HTTP service implementation, remote SDK, FFI and crate publication are outside this first implementation scope.

21.10 First implementation verification and completion criteria

Area	Required file-backend evidence
Normal use	Open/create a Hub, register the selected ontology/profile, commit Markdown and graph records, query them, create a baseline and reopen it after restart.
Denied and invalid changes	Reject unauthorised scope, invalid types/endpoints, prohibited graph structures, inconsistent ownership and malformed imports without changing the accepted head.
Concurrency	Race changes from the same base; accept a valid commit and return an explicit stale-head conflict for the other without mixed or lost revisions.
Replay	Retry the same operation and recover its original result; reject reuse for a different request fingerprint.
Recovery	Interrupt staging, content persistence, head publication and acknowledgement; reopen into a complete accepted snapshot and resolve uncertain commits through their original identities.
Portability	Export/import the selected model and verify exact IDs, revisions, relationships, Markdown content, source anchors, hashes, review records and declared recovery history.
Query consistency	Verify documented filter/order/cursor/traversal semantics and snapshot-bound reads, including stale derived-index reporting where applicable.
Rust/API boundary	Demonstrate use through public contracts without exposing file layout or native storage objects; record toolchain, executor and tested host/filesystem profiles.

The same conformance scenarios form the acceptance contract for future adapters. First-release results can establish only file-backend support; they must not be described as actual MongoDB or PostgreSQL parity.

Completion of this proposed first implementation requires working public Rust operations, actual file durability/recovery evidence, documented interchange and compatibility rules, and an independently inspectable example model. This section proposes that implementation; it does not create a crate, install a package, publish a release or change a system's implemented/accepted status.